Gamebot
객체 탐지 모델을 Android 조작 앱으로 출시

게임의 보상 확인·팝업 닫기·가방 정리 같은 반복 조작을 줄이려고 만들었습니다. **2020-2021년 AI 코딩 에이전트 없이 혼자 데이터 준비·학습·변환·Android 연동·출시까지 연결했고, 이후 서비스를 종료했습니다.** 기존 Darknet·TensorFlow Lite 프로젝트를 실제 게임 데이터와 모바일 환경에 맞게 통합한 경험입니다.

오프라인: 영상 → 프레임·라벨 → Darknet 학습 → SavedModel·TFLite. 기기: 캡처 → 전처리 → 객체 탐지 → 후보 정리 → 상태·행동 → 좌표 복원 → 접근성 입력.

1. 실제 화면을 학습·배포 가능한 자료로 만들기

녹화 영상의 프레임 추출 코드와 LabelImg·Yolo_mark를 이용해 라벨을 만들고, 게임·화면별 자료와 Darknet cfg를 준비했습니다. 직접 학습한 가중치를 SavedModel·TFLite로 변환했습니다. Darknet 실행 환경 질문과 자체 모델의 TensorFlow 추론 질문은 당시 직접 겪은 시행착오의 외부 기록입니다.

모델을 assets에 넣는 것만으로 끝나지 않았습니다. TFLite 무압축 패키징과 자산의 offset·length를 이용한 읽기 전용 mmap을 맞춰 Android Interpreter에 연결했습니다. mmap 사용을 메모리 감소율로 환산하지는 않습니다.

2. 모델의 후보와 실제 행동을 나누기

bbox·클래스 점수 tensor를 해석한 뒤 라벨별 신뢰도 기준과 클래스별 IoU NMS로 겹친 후보를 정리합니다. 남은 탐지를 직접 작성한 Target 상태 규칙으로 넘깁니다. 모델의 탐지와 게임의 업무 규칙을 같은 단계로 묶지 않았습니다.

홈 버튼은 여러 화면에 함께 나타납니다. 추론 시간에 비례한 관측 창의 반복 일치 검사로 잠깐 스친 화면과 지속된 상태를 구분하고, 이전 행동에 따른 후속 행동 선택으로 분해·확인·결과·복귀를 연결했습니다. 게임 UI가 바뀌면 모델과 규칙을 함께 수정해야 합니다.

3. 현재 화면과 실제 터치 좌표를 일치시키기

오래된 화면의 버튼을 뒤늦게 누르지 않도록 ImageReader 한 장 버퍼와 직렬 추론을 사용했습니다. 활성 경로는 Bitmap을 직접 전달하며 중간 JPEG 저장·재읽기를 거치지 않습니다. pixelStride·rowStride·padding을 반영한 캡처 뒤 모델 좌표를 실제 화면으로 되돌렸습니다.

회전 때 캡처 크기와 좌표 변환을 다시 구성하고, 자기 overlay 영역으로 향하는 터치를 막았습니다. 접근성 서비스가 실제 입력을 맡습니다. 전후 FPS·배터리 개선의 정량 수치는 별도 재현 자료가 없어 제시하지 않습니다.

4. 공통 엔진과 게임별 앱의 경계

공통 Android 라이브러리가 캡처·추론·좌표·입력·서비스 자원을 맡고, 게임 앱은 공통 Activity를 상속합니다. 입력 크기·출력 규격은 Gradle BuildConfig로 나눴습니다. 실제 모델 파일명과 일부 게임 규칙은 공통 코드에 고정된 부분이 남아 있습니다.

종료 때 추론 thread·TFLite·ImageReader·VirtualDisplay·MediaProjection·receiver·overlay를 정리했습니다. 게임별 패키지·모델 자산·버전·서명·최적화 설정으로 출시 단위를 관리했습니다.

실행과 남은 범위

바람의나라 실행 영상 · 라오킹 실행 영상 · 왕좌의게임 실행 영상.

영상은 Gamebot 계열 실행 사례입니다. 독립 녹화·기기·게임 버전의 정확도 재현과 현재 Android 전체 빌드 상태는 별도 과제입니다. AI 에이전트 미사용·1인 개발·출시 후 종료는 작성자 확인과 당시 소스·외부 질문을 함께 제시합니다.